1)Copy constructor:

It is a constructor that copies an object by creating a new empty object then filling it with the same values as the chosen object

Ex:

```
public Employee(Employee e){
Name = e.Name;
Id= e.Id
}
```

Then we can use it:

```
Employee e2 = new Employee(e1);
```

---

2)indexer

It lets you use object as an array of elements

It is useful in business when you have deal with a group of elements in each object

ex: shopping cart

ex:

```
class Classroom
{
   private string[] students = new string[3];

   public string this[int index]
   {
      get { return students[index]; }
      set { students[index] = value; }
   }
}
```

Then we are free to use it:

```
Classroom c = new Classroom();

c[0] = "Mariam";

c[1] = "Ahmed";

Console.WriteLine(c[0]);
```

3)

Struct: class-like but it is a value type , so it is passed by value, cant inherit from any class or struct

Access modifiers

Private: acc. Inside scope{} , default in class and struct

Private protected: acc. Inside scope + acc. In classes inherits from it within the same project

Protected: acc. in classes inherit from it

Internal: acc. inside the same project

Internal protected: acc. inside project + acc. in classes inherit from it

Public: acc. anywhere

Ctor: constructor. It initializes the values after creating the object with new

Override: changing the function implementation in a different class/struct

Encapsulation: getter/setter to achieve maintainability and scalability

Propfull: full property. It has the same performance as a normal getter/setter separated codes but it enhances readability

Prop: auto property. It is a shortcut to the full property and it still has the same performance but with better readability